```
$name = mb_convert_encoding("\u{C7}a\u{11F}ari", 'utf16');
var_dump(substr_count($name, 'a'));
var_dump(substr_count($name, "\u{C7}"));

var_dump(bin2hex($name));
var_dump(bin2hex('a'));
var_dump(bin2hex("\u{C7}"));

Output:
int(2)
int(0)
string(24) "00c70061011f006100720069"
string(2) "61"
string(4) "c387"
```

There are a few things to unpack here. First of all, we search for `'a'` and get a count of *2*. Secondly, we search for the Unicode character at codepoint U+00C7 (the Turkish letter Ç) and get a count of *0*. Finally, we get a dump of byte values for the UTF-16 string `$name`, then for the sake of this explanation, the same dump for `'a'` and `"\u{C7}"`. Can you spot what's going on?

We found `'a'` simply because the UTF-16 encoding is a two-byte sequence *0x00 0x61* (told you there were redundant bytes in fixed-width encodings), the ASCII (and therefore UTF-8) encoding for `'a'` is *0x61*, and that byte value does indeed appear twice in the `$name` byte stream. We get lucky because UTF-16 doesn't deviate too much from the ASCII range character byte values, and `substr_count()` is completely agnostic of encoding. Things aren't so great when we move outside the ASCII range. U+00C7 is encoded quite differently between UTF-16 and UTF-8 (*0x00 0xC7* and *0xC3 0x87*, respectively), and we get a count of *0*. The problem works both ways:

```
$name = mb_convert_encoding("\u{C387}", 'UTF-16'); // U+C387 is a Korean
                                                      Hangul Syllable
var_dump(substr_count($name, "\u{C7}")); // U+00C7 is a Latin Capital C
                                            with Cedilla

Output:
int(1)
```